

PRACTICAL 1 — Command History Buffer

```
GNU nano 7.2 history.c
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 100

int main()
{
    char history[MAX][SIZE];
    int count = 0;
    int choice;
    char command[SIZE];

    while (1)
    {
        printf("\n1. Store Command");
        printf("\n2. Display History");
        printf("\n3. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);
        getchar();

        if (choice == 1)
        {
            if (count == MAX)
            {
                printf("History is full!\n");
            }
            else
            {
                printf("Enter command: ");
                fgets(command, SIZE, stdin);

                command[strcspn(command, "\n")] = '\0';

                if (strlen(command) == 0)
                {
                    printf("Invalid command!\n");
                }
                else
                {
                    strcpy(history[count], command);
                    count++;

                    printf("Command stored successfully.\n");
                }
            }
        }
    }
}
```

```
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ nano history.c
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ gcc history.c -o history
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ ./history

1. Store Command
2. Display History
3. Exit
Enter choice: 2

Command History:

1. Store Command
2. Display History
3. Exit
Enter choice: 2

Command History:

1. Store Command
2. Display History
3. Exit
Enter choice: 1
Enter command: ls
Command stored successfully.

1. Store Command
2. Display History
3. Exit
Enter choice: 1
Enter command: pwd
Command stored successfully.

1. Store Command
2. Display History
3. Exit
Enter choice: 3
Exiting...
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ nano history.c
saireddy@DESKTOP-HKQRL9G:~/History_Practical$
```

PRACTICAL 2 — Pipeline using pipe()

```
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ ls | wc -l
2
saireddy@DESKTOP-HKQRL9G:~/History_Practical$
```

```
GNU nano 2.9.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t p1, p2;

    pipe(fd);

    p1 = fork();

    if (p1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", NULL);
    }

    p2 = fork();

    if (p2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}
```

```
saireddy@DESKTOP-HKQRL9G:~/History_Practical$ cd ..
mkdir Pipeline_Practical
cd Pipeline_Practical
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ nano pipeline.c
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ gcc pipeline.c -o pipeline
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ ./pipeline
2
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ |
```

PRACTICAL 3 — Create Pipes and Redirect Streams

```
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ echo "Hello Linux" | wc -c
12
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ |
```

```
saireddy@DESKTOP-HKQRL9G:~/Pipeline_Practical$ cd ..
mkdir Pipe_Redirect
cd Pipe_Redirect
```

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t p1, p2;

    pipe(fd);

    p1 = fork();

    if (p1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("echo", "echo", "Hello Linux", NULL);
    }

    p2 = fork();

    if (p2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("wc", "wc", "-c", NULL);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}

```

```

sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ nano redirect_pipe.c
sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ gcc redirect_pipe.c -o redirect_pipe
sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ ./redirect_pipe
12

```

PRACTICAL 4 — Multiple Pipes

```

sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ nano pipeline.c
sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ ls | grep .c | wc -l
3

```

```

sairedy@DESKTOP-HKQRL9G:~/Pipe_Redirect$ cd ..
mkdir MultiPipe_Practical
cd MultiPipe_Practical

```

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    if (fork() == 0)
    {
        // Process 1: ls
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);
    }

    if (fork() == 0)
    {
        // Process 2: grep
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);
    }

    if (fork() == 0)
    {
        // Process 3: wc
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);

    return 0;
}
```

```
saireddy@DESKTOP-HKQRL9G:~/MultiPipe_Practical$ nano multipipe.c
saireddy@DESKTOP-HKQRL9G:~/MultiPipe_Practical$ gcc multipipe.c -o multipipe
saireddy@DESKTOP-HKQRL9G:~/MultiPipe_Practical$ ./multipipe
1
saireddy@DESKTOP-HKQRL9G:~/MultiPipe_Practical$ |
```